Super Intelli Machines

Building agents that do the work, so people do not have to.

Document: Senior Engineering Interview Assignment
Confidential: Candidate Use Only
Version: 2.0

Take-Home Assignment: School Operations Agent Platform

A senior-level product slice: registrations, role-aware access, document ingestion, intent identification, multi-channel workflows, event-driven UI, and production-shaped operations.

Role: Senior / Staff Engineer | **Duration:** 4-6 days | **Stack:** Your choice | **Bar:** Ownable by a small team

1. What We Are Evaluating

This is not a feature checklist. We are evaluating whether you can turn an ambiguous real-world school workflow into a production-shaped system that a junior engineer could safely extend. The system should demonstrate product judgement, domain modeling, security boundaries, model orchestration, failure handling, and operational clarity.

- Can you identify the real domain entities and relationships without being handed a schema?
- Can you parse messy documents and messages into structured, auditable system actions?
- Can you separate model reasoning from deterministic business rules?
- Can you build registration, login, authorization, and scoped access without overbuilding enterprise SSO?
- Can you design the system so retries, duplicates, wrong-context actions, and model failures do not corrupt state?

2. Product Brief

Build a **School Operations Agent Platform** that helps a school manage classes, teachers, students, assignments, document-based instructions, reminders, submissions, and parent/guardian visibility. Telegram or WhatsApp may be used as the primary chat channel. A web app is required for registration, login, administration, document management, and operational visibility.

2.1 Domain Model Expectations

The system should reflect a real school structure. You may choose exact table names and boundaries, but the relationships must be clear and enforced.

- A school has grades/classes, teachers, students, parents/guardians, and administrators/coordinators.
- Teachers can belong to multiple classes. A class may have multiple teachers.
- Students belong to a school and grade/class, and may have one or more linked guardians.
- Assignments can target a class, a group, or an individual student.
- Documents may belong to a school, class, teacher, assignment, or student record.

- Messages, reminders, submissions, feedback, model decisions, and audit events must be traceable to the correct actor and resource.

2.2 Actors and Access

Actor	Capabilities	Access Boundary
School Admin / Coordinator	Register school, invite teachers, configure grades/classes, upload policy or curriculum documents, view school-level operations.	Only their school.
Teacher	Create assignments, upload class material, review student progress, approve reminders, send feedback.	Only assigned classes/students/resources.
Student	Receive work, ask questions, submit work, view feedback, view own dashboard.	Only own assignments, submissions, and feedback.
Parent / Guardian	Opt in, view high-level progress, receive digest/escalation messages.	Only linked child/children, with limited detail.
System / Agent	Interpret intent, parse documents, schedule reminders, summarise status, route messages.	Must act through explicit tools and policy checks.

2.3 Registration, Login, and Linking

- School admin/coordinator can register a school and create initial grades/classes.
- Teachers can be invited and log in to the web app.
- Students and guardians can be invited with scoped, short-lived links or codes.
- Chat identities must be linked to the correct web account or school record before sensitive actions are allowed.
- Sessions should be protected with practical web security: server-side authorization checks, token/session expiry, logout, and route protection.
- The system should prevent obvious cross-school or cross-class mistakes during onboarding.

3. Document Reading and Intent Identification

Senior candidates must implement document understanding, not just chat commands. The school should be able to upload documents that influence system behaviour and assignment creation.

3.1 Document Types

- Assignment brief: PDF, DOCX, image, or pasted text containing instructions and deadline details.
- Class roster: CSV or spreadsheet-style text containing students, grade/class, parent contact, and optional notes.
- School policy: reminder quiet hours, escalation rules, allowed channels, teacher approval requirements.
- Student submission: text, file/photo, or voice note transcript.

3.2 Parsing Expectations

- Extract structured fields such as title, subject, instructions, due date, target students/classes, attachments, and constraints.
- Handle missing or ambiguous fields by asking a follow-up question rather than guessing silently.
- Show extracted values for teacher/admin review before committing high-impact actions.
- Store the original document, parsed output, confidence/ambiguity notes, and user approval state.
- Protect against prompt injection inside uploaded documents.

3.3 Intent Identification

Incoming messages and uploaded documents should be routed through a clear intent layer. We expect the system to distinguish at least these intents:

- Create assignment, update assignment, cancel assignment.
- Progress update, blocked/help request, submission, resubmission.
- Teacher feedback, revision request, completion decision.
- Parent opt-in, parent digest request, escalation acknowledgement.
- Admin configuration change, roster import, policy upload.
- Unknown/unsafe/out-of-scope message.

4. Required Product Capabilities

1. School registration and login flow for coordinator/admin users.
2. Teacher invitation, login, role assignment, and class/grade assignment.
3. Student/guardian onboarding with scoped linking to the correct school/class/student record.
4. Document upload and parsing flow with review/approve before creating assignments or policies.
5. Natural-language and document-based assignment creation for individual, group, or class targets.
6. Chat-based progress updates, blocked state handling, submissions, resubmissions, and feedback.
7. Reminder engine with quiet hours, escalation rules, and manual trigger for demo.
8. Teacher/admin dashboards with live updates for submissions, blocked students, overdue assignments, and parse approvals.
9. Student dashboard showing own assignments, deadlines, submissions, feedback, and current status.
10. Audit/event timeline for important actions: registration, login, invite, parse, approve, assign, remind, submit, feedback, access denial, model failure.

5. Production-Readiness Expectations

> The implementation can be small, but the boundaries should be production-shaped. We care more about correct shape and deep reasoning than about many shallow features.

- **Authorization:** server-side checks for every school/class/student scoped action.
- **Idempotency:** webhook retries, double form submits, and repeated file uploads should not duplicate business actions.

- **State machine:** assignment and submission state transitions should be explicit and defensible.
- **Model boundaries:** LLM output should be structured, validated, and treated as proposed action until approved when needed.
- **Observability:** structured logs, correlation IDs, and enough event history to debug a wrong reminder or bad parse.
- **Security:** no secrets in git, safe upload handling, route protection, input validation, safe CORS, and reasonable rate limits.
- **Privacy:** logs and dashboards should avoid exposing unnecessary student/guardian personal data.
- **Recovery:** scheduler restart and failed model/API calls should not leave assignments stuck silently.

6. Minimum Live Scenario

During the interview, we will exercise a scenario like this:

1. Register a school, create two grades/classes, invite one teacher, two students, and one guardian.
2. Upload a roster document and resolve at least one ambiguous or duplicate row.
3. Upload an assignment brief. The system parses it, identifies missing/ambiguous details, asks for clarification, and creates an assignment after approval.
4. One student reports progress. Another student says they are blocked. The teacher dashboard updates.
5. The reminder job runs. It respects quiet hours/policy and treats silent, blocked, and submitted students differently.
6. A student submits work as text or file/photo. Teacher gives feedback and requests revision or completes the assignment.
7. A wrong-context access attempt is rejected or safely guided.
8. The audit/event timeline explains what happened without requiring database inspection.

7. Required Deliverables

#	Deliverable	Notes
1	Web App	Registration, login, role-aware dashboards, document upload/review, assignment and audit views.
2	Chat Channel	Telegram or WhatsApp working end-to-end; bonus for both.
3	Domain Model	School, grades/classes, teachers, students, guardians, assignments, documents, submissions, reminders, feedback, audit events.
4	Document Parser	At least two document types, with parsed output, confidence/ambiguity notes, and approval workflow.
5	Intent Layer	Clear routing for messages/documents into system actions, including unknown/unsafe fallback.
6	Auth and Authorization	Registration/login, scoped invites, session handling, role/resource checks, protected routes.
7	Reminder/Scheduler	Policy-aware reminders, manual trigger for demo, restart-safe enough for the exercise.
8	Live Updates	WebSocket/SSE or comparable approach for teacher dashboard updates.
9	Tests/Evals	Auth/policy tests plus at least one document parsing or intent identification eval.
10	README + ADRs	Setup, assumptions, architecture diagram, domain/access model, parser strategy, known limitations, and 2-3 short ADRs.

> **Out of scope:** enterprise SSO, full billing, native mobile apps, complex LMS integrations, multi-region deployment, and polished visual design. Do not skip registration, authorization, parsing, or auditability; those are central to this senior assignment.

8. Evaluation Rubric (100 points)

Area	Weight	What we look for
Product and Domain Judgement	15	Good school/class/user relationships and practical handling of ambiguity.
Document Parsing and Intent Layer	15	Structured extraction, review workflow, follow-up questions, unsafe/unknown fallback.
Auth and Authorization	15	Registration/login, protected routes, scoped invites, role/resource checks, wrong-context handling.
End-to-End Correctness	15	The minimum scenario works live without manual database edits.
State, Data, and Idempotency	10	Clear lifecycle, persistence, audit events, duplicate handling, restart behaviour.
Model/Agent Architecture	10	Clean model boundaries, validated structured outputs, tool/action separation.
Operational Readiness	8	Logs, correlation IDs, safe uploads, retries/fallbacks, runbook quality.
UI Usefulness	5	Role-aware dashboards make state and required actions obvious.
Tests and Evals	5	Meaningful coverage of auth/policy and parsing/intent behaviour.
Communication and Mentorship	2	Readable docs, clear trade-offs, code a junior can extend.
Total	**100**	

Bonus (up to +15 points)

- Support both Telegram and WhatsApp with a canonical message envelope.
- OCR or voice-note transcription.
- Document chunking/retrieval for asking questions against uploaded school policy or assignment material.
- Feature flags for model/prompt versions or reminder policies.
- OpenTelemetry or trace-style visualization across channel -> intent -> action -> notification.
- One-page threat model covering prompt injection, upload abuse, wrong-recipient messages, and privacy leakage.

9. Ground Rules

1. **Commit intentionally.** We will review history. A single final commit is a concern at this level.
2. **Declare templates or starter repos.** Framework scaffolds are fine; undisclosed copied work is not.
3. **Keep secrets out of git.** Provide `.env.example` and document secret rotation assumptions.
4. **Prefer depth over breadth.** A complete, defensible slice beats many shallow stubs.

5. **Make trade-offs explicit.** Document what you simplified and how you would productionize it.

10. Submission

Email the following to **Uma +91 88866 77288** before the agreed deadline:

1. GitHub repo URL.
2. Deployed URL or local run instructions.
3. Chat bot handle/test number if applicable.
4. 8-12 minute demo video covering registration, document parsing, intent handling, reminder, submission, feedback, access denial, and audit trail.
5. Short implementation note, up to 700 words: assumptions, domain/access model, parsing strategy, state model, failures handled, and what you would improve next.

11. Interview Round

Time	Activity
10 min	Live demo of the minimum scenario.
20 min	Architecture walkthrough: domain model, access model, parser, intent layer, scheduler.
20 min	Pair extension: add a new document type, new role, new access rule, or new intent.
10 min	Mentorship simulation: explain how a junior should safely add a feature.

> **Questions?** Reach out to **Uma +91 88866 77288**. Smart clarifying questions and scope-pruning proposals are welcome.
